File Handling in Python

- What is File Handling
- Open & Read a File
- Write & Append to a File
- Create a File
- Close a file
- Work on txt, csv, excel, pdf files

File handling in Python

File handling in Python allows you to read from and write to files. This is important when you want to store data permanently or work with large datasets.

Python provides built-in functions and methods to interact with files.

Steps for File Handling in Python:

- Opening a file
- Reading from a file
- Writing to a file
- Closing the file

Open a File

To perform any operation (read/write) on a file, you first need to open the file using Python's open() function.

Syntax: `file_object = open('filename', 'mode')`

- ***'filename***': Name of the file (can be relative or absolute path).
- ***'mode***': Mode in which the file is opened (read, write, append, etc.).

File Modes:

- 'r': Read (default mode). Opens the file for reading.
- 'w': Write. Opens the file for writing (if file doesn't exist, it creates one).
- 'a': Append. Opens the file for appending (if file doesn't exist, it creates one).
- 'rb'/'wb': Read/Write in binary mode.

Example: Opening a file for reading

```
file = open('example.txt', 'r')
```

Read from a File

Once a file is open, you can read from it using the following methods:

- read(): Reads the entire content of the file.
- readline(): Reads one line from the file at a time.
- readlines(): Reads all lines into a list.

```
# Example: Reading the entire file
file = open('example.txt', 'r')
content = file.read()
print(content) file.close()
```

```
# Example: Reading one line at a time
file = open('example.txt', 'r')
line = file.readline() print(line)
file.close()
```

Write to a File

To write to a file, you can use the write() or writelines() method:

- write(): Writes a string to the file.
- writelines(): Writes a list of strings.

```
# Example: Writing to a file (overwrites existing content)
file    =    open('example.txt',    'w')
file.write("Hello,                world!")
file.close()
```

Example: Appending to a file (add line to the end)

```
file = open('example.txt', 'a') file.write("\nThis
is an appended line.") file.close()
```

 # Close a file:

```
file.close()
```

Close a File

Instead of manually opening and closing a file, you can use the **with** statement, which automatically handles closing the file when the block of code is done.

 # Example: Reading with `with` statement

```
with open('example.txt', 'r') as file:
    content = file.read()
print(content)
```

In this case, you don't need to call file.close(), because Python automatically closes the file when the block is finished.

*# Example: Using **exception handling** to close a file*

```
try:
    open('example.txt', 'r') as file:
     content = file.read()
print(content) finally:
    file.close()
```

Working with Difft Format Files

 # csv - Using csv module

```
import csv  file =
open('file.csv', mode='r')
reader = csv.reader(file)
```

 # csv - Using pandas library

```
import pandas as pd  df =
pd.read_csv('file.csv')
```

 # excel - Using pandas library

```
import pandas as pd  df =
pd.read_excel('file.xlsx')
```

 # PDF Using PyPDF2 library:

```
import PyPDF2  file =
open('file.pdf', 'b')
pdf_reader = PyPDF2.PdfReader(file)
```

CSV File Operations (Optional but useful)

Write CSV using Python

```
import csv

with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Roll", "Name", "Marks"])
    writer.writerow([1, "Nikita", 90])
    writer.writerow([2, "Harshit", 88])
```

Read CSV

```
import csv

with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Convert TXT → CSV (Migration)

```
import csv

with open("students.txt") as src, open("students.csv", "w", newline="") as dest:
    writer = csv.writer(dest)
    for line in src:
        writer.writerow(line.strip().split(","))
```

FILE POINTER & NAVIGATION

1. File Pointer

A **file pointer** is the current position inside a file where reading/writing will happen next.

Whenever we open a file:

- For reading → pointer starts at **0**
- For writing with "w" → file is cleared and pointer starts at **0**
- For appending "a" → pointer starts at **end of file**

2. tell()

tell() → returns the **current position** of the file pointer (in bytes).

Example:

```
f = open("sample.txt", "r")
print(f.tell())      # Shows 0 initially
f.read(5)
print(f.tell())      # Shows 5
f.close()
```

3. seek(offset, whence)

Moves the file pointer to a new location.

Syntax

```
file.seek(offset, whence)
```

whence Values

Whence	Meaning	Example
0	From beginning	seek(5, 0)
1	From current position	seek(3, 1)
2	From end of file	seek(-5, 2) → 5 bytes before end

Example: Using seek() in a text file

```
f = open("demo.txt", "r")

print(f.read(10))
print("Pointer:", f.tell())

f.seek(0)    # move to start again
print(f.read())
```

```
f.close()
```

Example: Using seek() in a binary file

Binary files allow movement in any direction accurately.

```
f = open("image.jpg", "rb")

f.seek(50, 0)
data = f.read(20)
print(data)

f.close()
```

UPDATING SPECIFIC FILE CONTENT

You cannot directly “edit in the middle” of a text file.

Correct method:

1. Read entire file into memory
2. Modify the content
3. Rewrite the file

Example: Update one specific line

```
with open("students.txt", "r") as f:
    lines = f.readlines()

lines[1] = "Updated Line\n"

with open("students.txt", "w") as f:
    f.writelines(lines)
```

PROGRAMMING USING FILE I/O

Log Management Program

Log writing

```
def write_log(message):
    with open("app.log", "a") as f:
        from datetime import datetime
        time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        f.write(f"{time} : {message}\n")

write_log("Application Started")
write_log("User logged in")
```

Log reading with pointer movement

```
with open("app.log", "r") as f:
    print("Pointer:", f.tell())
    print(f.read(20))

    f.seek(0)
    print("\nFull Log:")
    print(f.read())
```

Student Record Management System

Features

- Add student
- Search student
- Delete student
- Update record

Data file format (CSV-like):

1, Nikita, MCA, 89
2, Harshit, B.Tech, 75

Code: Add + View Students

```
def add_student():
    roll = input("Enter roll no: ")
    name = input("Enter name: ")
    course = input("Enter course: ")
    marks = input("Enter marks: ")

    with open("students.txt", "a") as f:
        f.write(f"{roll},{name},{course},{marks}\n")

def view_students():
    with open("students.txt") as f:
        for line in f:
            print(line.strip())
```

Code: Search Student

```
def search_student(roll_no):
    with open("students.txt") as f:
        for line in f:
            data = line.strip().split(",")
            if data[0] == roll_no:
                return line
```

```
    return "Not found"
```

Code: Delete Student

```
def delete_student(roll_no):
    with open("students.txt") as f:
        lines = f.readlines()

    with open("students.txt", "w") as f:
        for line in lines:
            data = line.strip().split(",")
            if data[0] != roll_no:
                f.write(line)

    print("Record deleted successfully.")
```

Data Migration Program

Moving data from one file to another.

Example: Migrate only high-scoring students

```
with open("students.txt", "r") as src, open("toppers.txt", "w") as dest:
    for line in src:
        roll, name, course, marks = line.strip().split(",")
        if int(marks) >= 80:
            dest.write(line)
```